객체지향 프로그래밍 기초

10강. 포인터

포인터

- 데이터를 저장할 때 컴퓨터 프로그램이 알아야 할 세가지 기본 사항

  ①어디에 저장되는가?

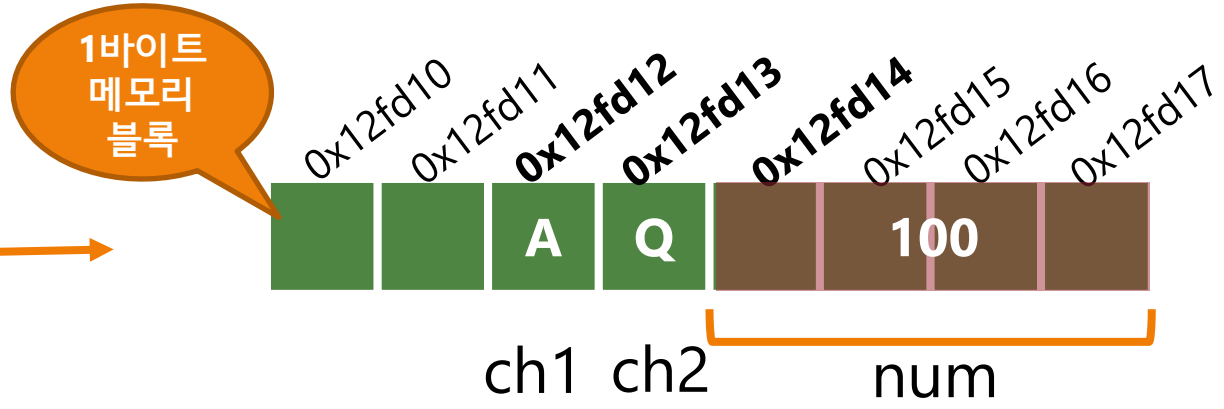
  ②어떤 값이 저장되는가?

  ③어떤 종류의 정보인가?

포인터

- **변수가 메모리상에 어떻게 존재하는가?**

```
int main()
{
    char ch1='A', ch2='Q';
    int num = 100;
    . . .
    return 0;
}
```



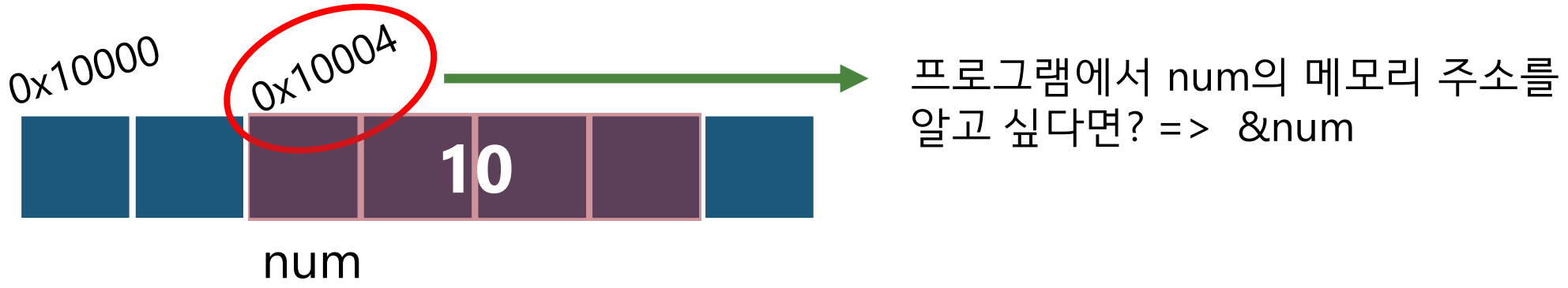
<변수의 선언과 메모리 할당>

- **num은 어디에 할당되어 있는가?**
  - "int형 변수 num은 0x12fd14번지에 할당되어 있다"(시작번지만을 가지고 위치를 표현)
  - 주소값 0x12fd14도 저장이 가능한 값이며, 주소를 저장하는 변수가 **포인터 변수**이다.

포인터

- 변수의 주소는 어떻게 알아내는가?
  - 주소연산자(&)를 변수 앞에 붙이면 그 변수의 주소를 알 수 있다.
    -& : 변수의 주소를 반환하는 연산자

int num = 10;



포인터

- 변수의 주소 알아내기

```
int main()
{
    int donuts =6;
    double cups = 4.5;
    cout << "도너츠 : " << donuts;
    cout << ", 도너츠의 주소 : " << &donuts << endl;
    cout << "컵 : << cups << ", 컵의 주소 : " << &cups <<endl;
    return 0;
}
```

포인터 선언

- 포인터는 어떤 값의 주소를 저장한다. 즉, 포인터의 이름이 주소를 나타낸다.
- 간접참조 연산자(*)를 포인터 이름 앞에 붙이면 그 주소에 저장되어 있는 값이 된다.
- 포인터를 선언할 때 그 포인터가 지시하는 데이터형이 무엇인지 서술해야 한다.

포인터 선언

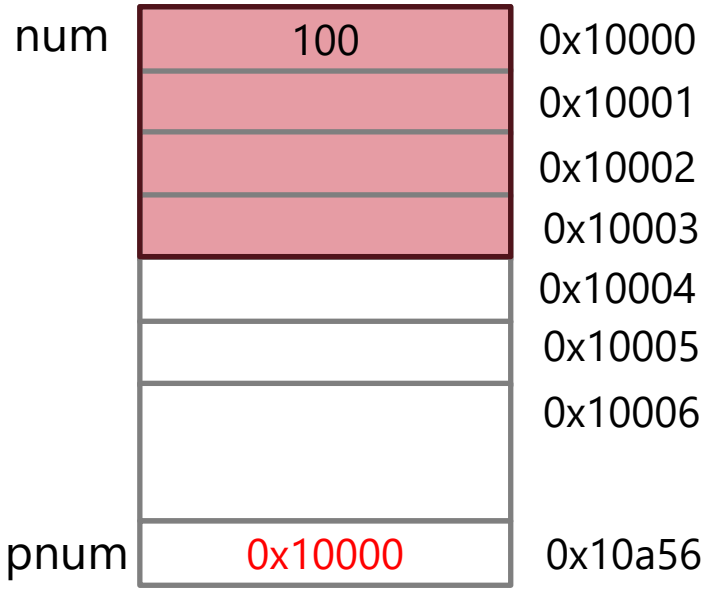
```
int   *   p_update;
```

- * p_updat가 int형이라는 것을 나타낸다.
- p_update의 데이터형은 int형을 지시하는 포인터, 즉 int * 형이다.
- p_update는 포인터(주소)이고, *p_update는 포인터가 아니라 int형 변수이다.

포인터에 값 대입

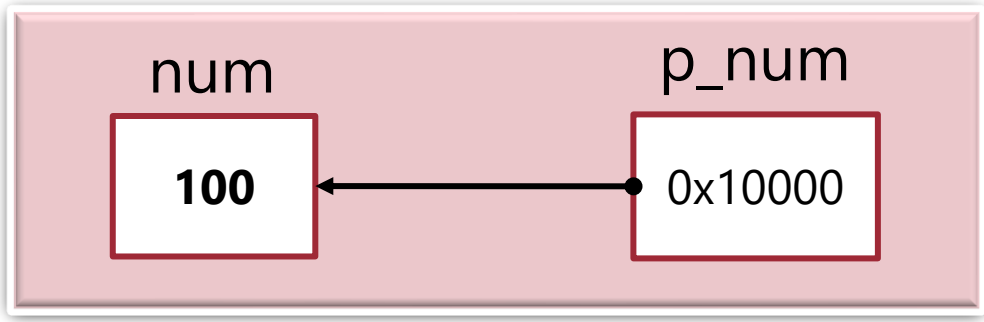
```
int main()
{
    int num = 100;
    int*  p_num;
    pnum = &num;

    return 0;
}
```



int * pnum; // int형 변수의 주소값을 저장하는
포인터 변수 선언
// pnum은 포인터 변수의 이름

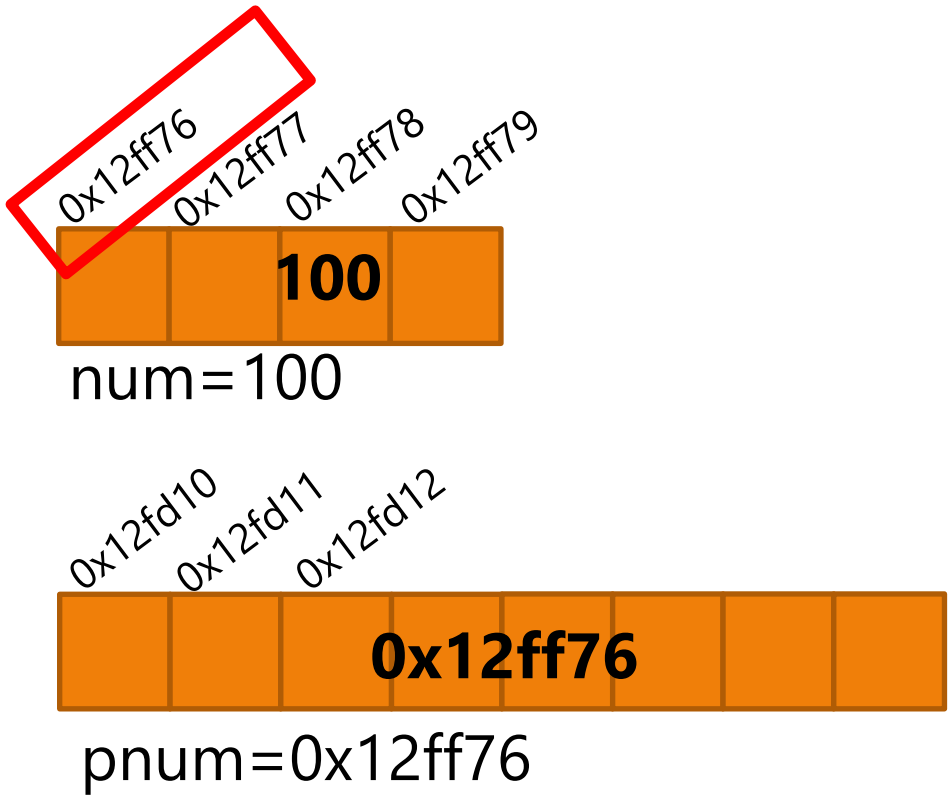
pnum = # // num의 주소값을 반환



포인터에 값 대입

```
int main()
{
    int num = 100;
    int *pnum;
    pnum = &num;

    return 0;
}
```

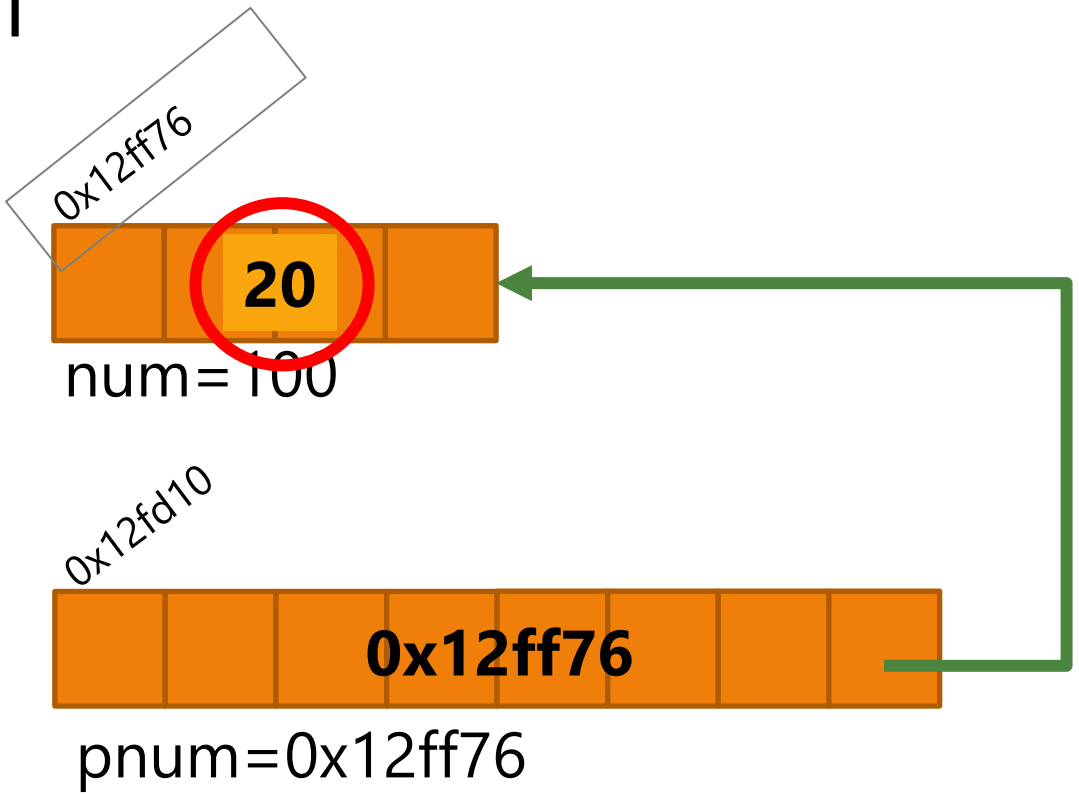


- **포인터 변수 pnum에는 변수 num의 메모리 시작번지값이 저장된다.**
- **"포인터 변수 pnum이 int형 변수 num을 가리킨다."**

포인터에 값 대입

- **포인터변수가 가리키는 변수의 값을 바꾸기**

```
int main()
{
    int num = 5;
    int *pnum = &num;
    *pnum = 20;
    cout << *pnum;
    return 0;
}
```



- *pnum : 포인터 pnum의 최종값.
  "포인터 pnum의 주소가 가리키는
  변수 num에 20을 저장하라 "

포인터 예제

```
int main()
{
    int update = 6;
    int * p_update;

    p_update = &update;
    cout << "값 : update = " << update;
    cout << ", p_update = " << *p_update << endl;

    cout << "주소 : update = " << &update;
    cout << " , p_update = " << p_update << endl;

    *p_update = *p_update + 1;
    cout << "변경된 update = " << update << endl;
    return 0;
}
```

포인터 초기화

```
int main()
{
    int num = 100;
    int *  pnum = &num;

    return 0;
}
```

포인터 초기화

- 포인터변수의 데이터형과 포인터가 가리키는 변수의 데이터형은 같아야 한다.

```
int main()
{
    int num = 5;
    int *pnum = &num;

    return 0;
}
```

```
int main()
{
    int num = 5;
    double *pnum = &num;

    return 0;
}
```

포인터의 간접 참조

- 포인터의 간접 참조는 포인터가 지시하는 주소에 저장되어 있는 값을 참조한다는 뜻이다.

```
cout << *pc
*pc = 120;
```

포인터 사용 방법

❖ 선언만 하고 초기화없이 참조값에 접근하여 사용하면 안된다.

```
int main()
{
    int * pnum;
    *pnum = 20;
    return 0;
}
```

X

❖ 포인터 변수에 사용자가 임의의 값을 할당하면 안된다.

```
int main()
{
    int * ptr1 = 125;
    *ptr1 = 100;
    return 0;
}
```

X

포인터 사용 방법

❖ 포인터 ptr1, ptr2는 아무 곳도 가리키지 않는다.(null pointer)

```
int main()
{
    int *ptr1 = 0;
    int *ptr2 = NULL;
    .....

}
```

포인터 예제

- 아래 프로그램은 어떤 부분이 잘못 되었는가?

```
int main()
{
    int num1 = 5;
    double *pnum1 = &num1;

    double num2 = 10;
    int *pnum2 = &num2;

    return 0;
}
```

X

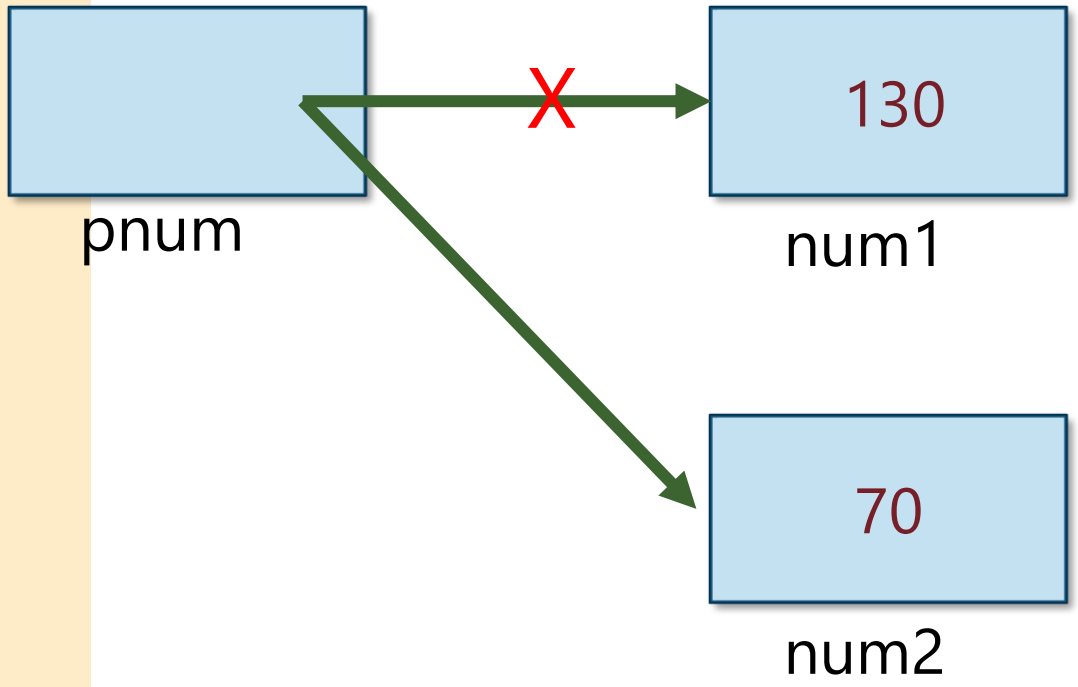
포인터 예제

- 아래 프로그램의 실행 결과는?

```
int main()
{
    int num1 = 100, num2 = 100;
    int *pnum;

    pnum = &num1;
    (*pnum) += 30;
    pnum = &num2;
    (*pnum) -= 30;

    cout << num1 << num2 << endl;
    return 0;
}
```



[예제1]

double형 변수 ted를 선언하고 3.5로 초기화하라.

ted를 가리키는 포인터를 선언하고, 그 포인터를 사용하여 ted의 값을 출력하라.

10강. 포인터

포인터, 배열, 포인터 연산

포인터 연산

- **정수형 변수에 1을 더하면 값이 1만큼 증가한다.**
- **포인터 변수에 1을 더하면 값은 그 포인터가 가리키는 데이터형의 바이트 수만큼 증가한다.**

```
double   num = 3.04;
double*  p_num = &num;
p_num = p_num+1;
```

-> 포인터 주소값에 8을 더하는 것과 같다.

```
double   num = 3.04;
double*  p_num;
*p_num = *p_num+1;
```

-> p_num이 가리키는 값(num의 값)에 1을 더하는 것과 같다.

포인터와 배열

- **포인터와 배열의 공통점**
  - 주소값을 갖는다.
  - 인덱싱(첨자접근)과 포인터 연산이 가능하다.

- **포인터와 배열의 차이점**
  - 배열명은 상수이다.
  - 배열과 포인터의 사이즈가 다르다.

포인터와 배열

- **배열 주소 확인하기**

```
int arr[3] = {0, 1, 2};

cout << "배열의 이름: " << arr << endl;
cout << "첫번째 요소: " << &arr[0]<< endl;
cout << "두번째 요소: " << &arr[1] << endl;
cout << "세번째 요소: " << &arr[2] << endl;
```

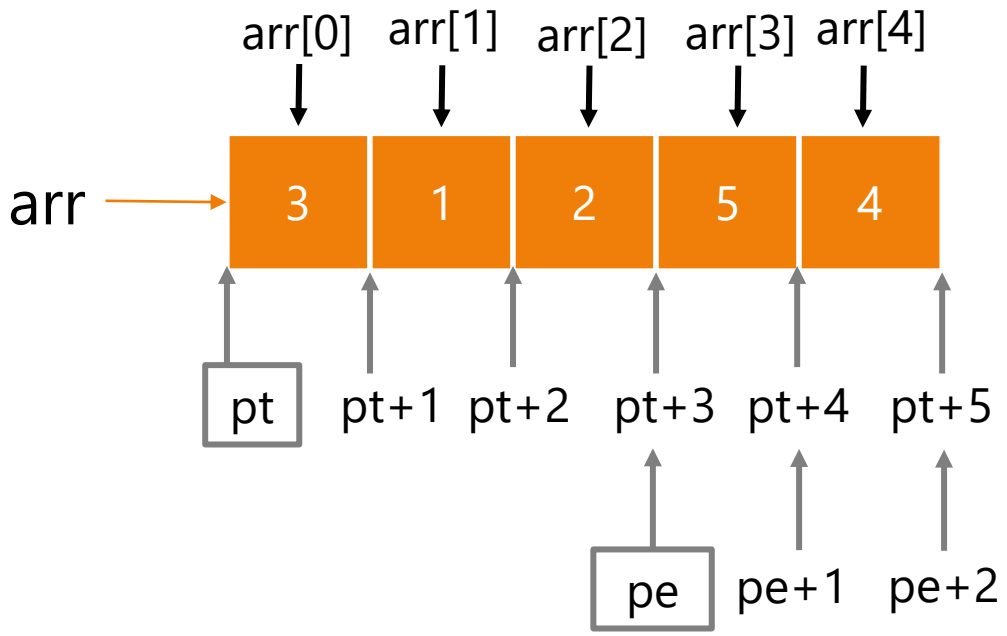


포인터와 배열

비교대상	포인터변수	배열의 이름
이름이 존재하는가?	존재한다	존재한다
무엇을 저장하는가?	메모리의 주소값	메모리의 주소값
주소값의 변경이 가능한가?	가능하다	불가능하다

포인터와 배열

```
int main()
{
    int arr[5] = {3, 1, 2, 5, 4};
    int * pt = arr;
    pt = pt + 1;
    int * pe = &arr[3];
    pe = pe - 1;
    return 0;
}
```



포인터와 배열

```
int main()
{
    int array[3] = {5, 10, 15};
    int *pt;

    pt = array;
    cout << “pt : “ << pt[0] << “ “ << pt[1] << “ “ << pt[2] << endl;
    cout << “array : “” << array[0] << “ “ << array[1] << “  “ << array[2] << “  “ <<endl;

    return 0;
}
```

[예제]포인터와 배열

```
int main()
{
    int arr1[3] = {1, 2, 3};
    double arr2[3] = {1.1, 2.2, 3.3};

    cout << *arr1 << setw(10) << *arr2 << endl;
    *arr1 += 100;
    *arr2 += 120.5
    cout << arr1[0] << setw(10) << arr2[0] << endl;

    return 0;
}
```

[예제]포인터와 배열

```
int main()
{
    int arr[3] = {11, 22, 33};
    int *ptr = arr;

    cout << *ptr << setw(10) << *(ptr+1) << setw(10) << *(ptr+2) << endl;
    ptr++;
    cout << *ptr << endl;
    ptr++;
    cout << *ptr << endl;
    ptr--;
    cout << *ptr << endl;

    return 0;
}
```

[예제2]

10개의 float형 원소를 가진 배열을 선언하고 임의의 데이터를 초기화하라.

첫번째 원소를 지시하는 포인터를 선언하고, 그 포인터를 사용하여 배열의 첫번째 원소와 마지막 원소를 출력하라.

[예제3] 포인터를 이용한 배열의 접근

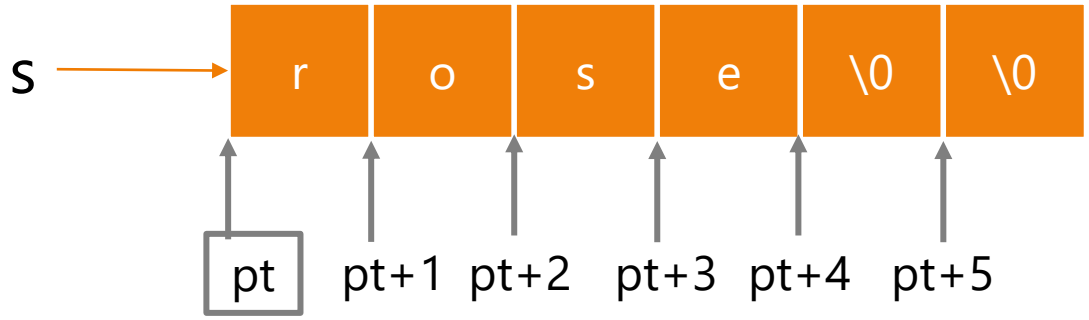
길이가 5인 int 배열 arr를 선언하고 이를 1, 2, 3, 4, 5로 초기화한 다음, 이 배열의 첫번째 요소를 가리키는 포인터 변수 ptr을 선언한다.
그 다음 포인터 변수 ptr만을 이용하여 모든 배열 요소의 값을 2씩 증가시키고, 출력하는 프로그램을 작성하시오.

[예제4] 포인터를 이용한 배열의 접근

길이가 5인 int 배열 arr를 선언하고 이를 1, 2, 3, 4, 5로 초기화한 다음, 이 배열의 마지막 요소를 가리키는 포인터 변수 ptr을 선언한다.
그 다음 포인터 변수 ptr만을 이용하여 배열에 저장된 모든 정수를 더하고 그 결과를 출력하는 프로그램을 작성하시오.

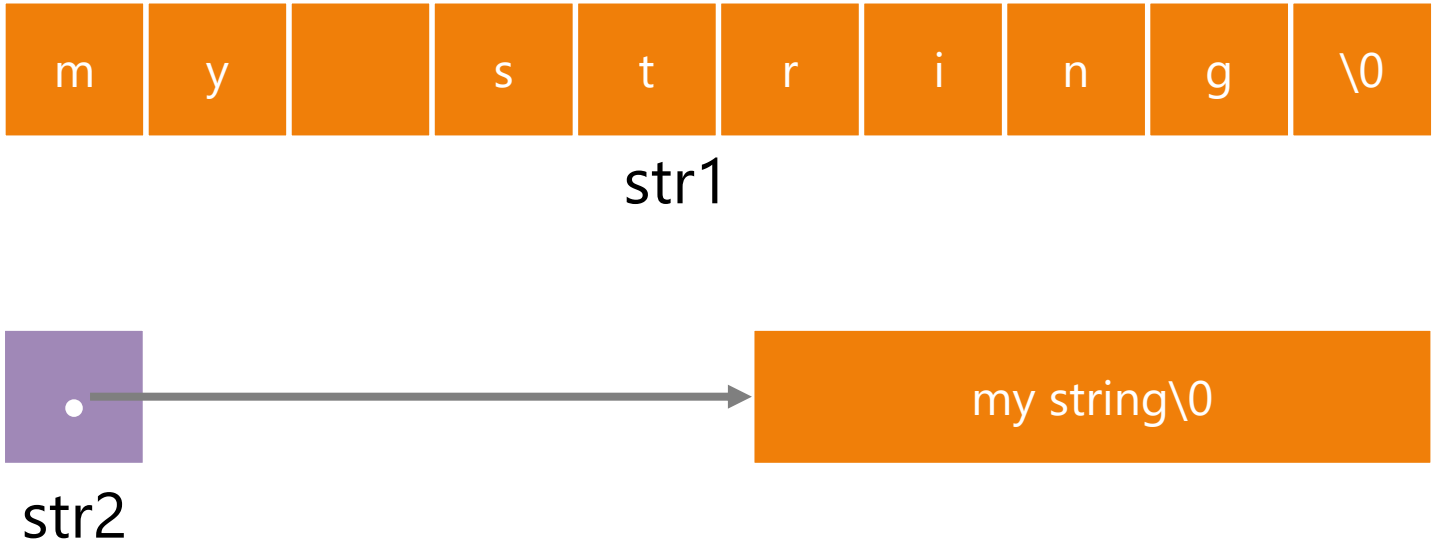
포인터와 문자열

```
char flower[10] = “rose”;
char * pt = flower;
cout << flower << “ and ” << pt << endl;
```



포인터와 문자열

```
char str1[ ] = “my string”;
const char *str2 = “my string”;
```



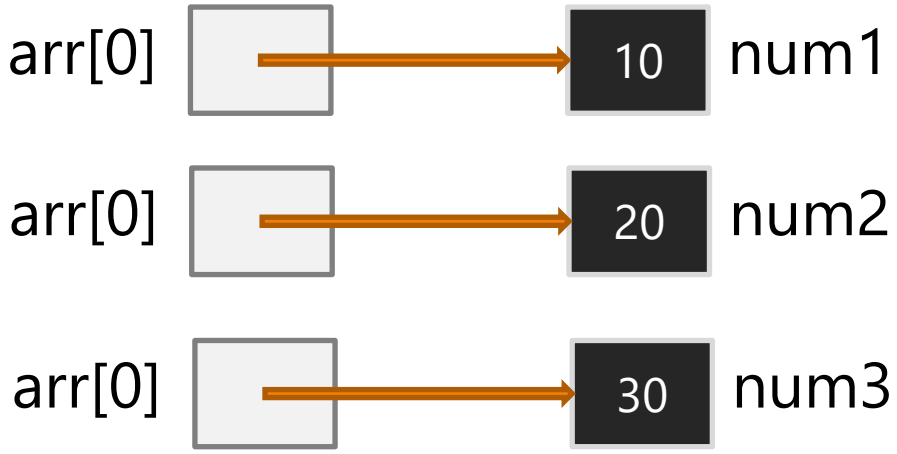
포인터 변수로 이루어진 배열 : 포인터 배열

```
int * arr1[20];          //길이가 20인 int형 포인터 배열 arr1
double * arr2[30];
```

포인터 배열

```
int main()
{
    int num1=10, num2=20, num3=30;
    int *arr[3]={&num1, &num2, &num3};

    printf("%d \n", *arr[0]);
    printf("%d \n", *arr[1]);
    printf("%d \n", *arr[2]);
    return 0;
}
```



문자열 포인터 배열

- 문자열을 저장하는 포인터 배열

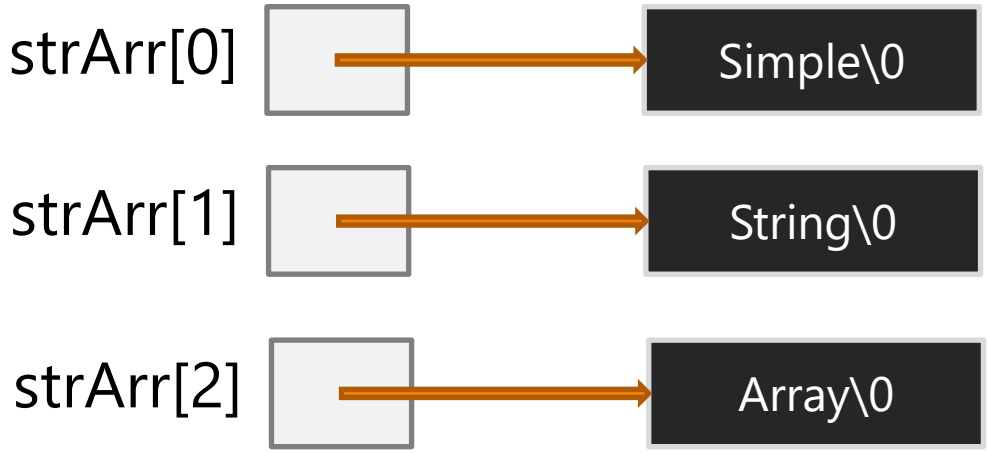
  char * strArr[3]; // 길이가 3인 char형 포인터 배열

문자열 포인터 배열

- 큰 따옴표로 묶여서 표현되는 문자열은 그 형태에 상관없이 메모리 공간에 저장된 후 그 주소값이 반환된다.

```
int main()
{
    char *strArr[3] = {“Simple”, “String”, “Array”};

    printf(“%s \n“, strArr[0]);
    printf(“%s \n“, strArr[1]);
    printf(“%s \n“, strArr[2]);
    return 0;
}
```



10강

포인터와 함수

매개변수의 전달 : Call by Value

```
double cube(double x);

int main()
{
    int side = 6;
    double volume = cube(side);
    cout << volume << endl;
    return 0;
}
double cube(double x)
{
    return x * x * x;
}
```

side라는 변수를 만들고 값 6을 대입한다.



원본

cube() 함수에 값 6을 전달한다.

x라는 변수를 만들고, 전달된 값 6을 대입한다.



복사본

매개변수의 전달 : Call by Value

- 두 매개변수의 값을 바꾸는 SwapValue() 함수를 호출하여 x, y 값을 바꾸고자 한다.
- 함수 호출 후, x, y 값이 변경되는가?

```
void SwapValue(int px, int py)
int main()
{
    int x = 3, y = 7;
    cout << "x = " << x << " y = " << y
<<endl;
    SwapValue(x, y);
    cout << "x = " << x << " y = " << y
<<endl;
    return 0;
}
```

```
void SwapValue(int px, int py)
{
    int temp;
    temp = px;
    px = py;
    py = temp;
}
```

매개변수의 전달 : Call by reference

- 매개변수로 주소를 전달한다.

```
void SimpleFunc(int *param);

int main()
{
    int    num = 10;
    int * p = &num;
    SimpleFunc(p);
    cout << num;
    return 0;
}
void SimpleFunc(int *param)
{
   param = 30;
}
```

매개변수의 전달 : Call by reference

```
void SimpleFunc(int *param, int len)
{
    int i;
    for (i=0;i<len;i++)
        param[i] = i;
     cout << endl;
}

int main()
{
    int arr[3] = {1, 2, 3};
    SimpleFunc(arr, 3);
    return 0;
}
```

[예제]Call by reference

```
void SwapValue(int *px, int *py);
int main()
{
    int x = 3, y = 7;
    cout << “x = “ << x << “ y = “ << y <<endl;
    SwapValue(&x, &y);
    cout << “x = “ << x << “ y = “ << y <<endl;
    return 0;
}

void SwapValue(int *px, int *py)
{
    int temp = *px;
    *px = *py;
    *py = temp;
}
```

[예제]Call by reference

- 변수의 주소값을 인자로 받아서 해당 변수에 직접 접근한다.

```
void SwapValue(int *px, int *py)
{
    int temp = *px;
    *px = *py;
    *py = temp;
}
```

매개변수의 전달

```
void usingArray(char arr[]);
int main()
{
    char str[ ] = “Hello World!”;
    usingArray(str);
    cout << str <<endl;
    return 0;
}

void usingArray(char arr[ ])
{
    arr[10] = ‘T’;
    arr[11] = ‘C’;
}
```

```
void usingArray(char * ps);
int main()
{
    char str[ ] = “Hello World!”;
    usingArray(str);
    cout << str <<endl;
    return 0;
}

void usingArray(char * ps)
{
    ps[10] = ‘T’;
    ps[11] = ‘C’;
}
```

[예제5]

배열을 매개변수(포인터를 사용)로 받아 배열의 값 중 최대값을 반환하는 함수 int maxNumber()를 정의하라.
maxNumber()를 이용하여 다음 배열의 최대값을 구하는 프로그램을 작성하라.

```
int array[10] = { 3,5,4,9,10,2,8,14,23,11};
```

10강. 포인터

포인터와 const

포인터와 const

- 포인터에 const를 사용하는 방법
  - 상수 객체를 가리키는 포인터를 만드는 것.

    상수 객체를 가리키는 포인터를 사용하여 그 포인터가 가리키는 값을 변경할 수 없도록 한다.
  - 포인터 자신을 상수로 만드는 것

포인터와 const

```
int age = 20;
const int * pt = &age;
*ptr=30;
num=40;
```

- *pt가 const int 형이므로 pt를 사용하여 그 값을 변경할 수 없다.
- 포인터의 const선언은 값을 변경하는 방법에 제한을 두는 것이지 무엇인가를 상수로 만드는 선언은 아니다.
- "포인터 변수 pt가 가리키는 변수 age가 상수화되는 것은 아니다. 따라서 변수 age에 저장된 값을 변경하는 것은 허용이 된다 "

포인터 변수의 상수화

```
int num1=20, num2=30;
int * const ptr = &num1;
ptr=&num2;
*ptr=40;
```

- 포인터 변수 ptr은 상수가 된다. 즉, 한번 주소값이 저장되면 주소값의 변경이 불가능하다.
- ptr이 상수일 뿐이므로 ptr이 가리키는 대상에 저장된 값을 변경하는 연산은 문제가 되지 않는다.